Finance Analytics Dashboard

Phase 3: Project Abstract and Development Reflection

Finance Analytics Dashboard – GitHub Repository

1. Introduction

The Finance Analytics Dashboard is a full-stack web application that has been designed to help users explore financial transactions and understand their financial activity through interactive tables and visual analytics. The project's objective was to demonstrate frontend and backend integration, RESTful API communication, relational database management, and dynamic financial analysis. The application contains two main views: Transactions, which presents financial records in an interactive table, and Analytics, which presents financial information through a pie chart of expenses by category, a bar chart comparing monthly income and expenses, and a line chart showing spending trends over time.

2. Methods and Technologies

Development followed the project phases of conceptual design, implementation, and finalization. The frontend was built with HTML5, CSS3, JavaScript, Bootstrap 5, DataTables.js, and Chart.js. DataTables.js supports searching, sorting, filtering, and server-side pagination. Chart.js provides the financial visualizations.

The backend uses Python and Flask to expose REST API endpoints for transaction records and analytical data, while MySQL stores information in related categories and transactions tables. SQL techniques including SUM(), CASE expressions, GROUP BY, and JOIN operations support the calculation of financial summaries. AJAX requests allow the frontend to retrieve data and update interface components.

3. Results

The application integrates the frontend, Flask API, and MySQL database to provide interactive transaction browsing and financial visualizations. The backend provides endpoints for retrieving transactions, monthly summaries, category summaries, and spending trends. Testing confirmed that the transaction API supports searching, sorting, pagination, and filtering by income and expense type. The three dashboard charts display data correctly, and the analytical endpoints returned data for the corresponding visualizations. Additional refinements included improving the DataTables response, displaying the actual transaction count instead of an estimate, and adding a validation endpoint. This endpoint checks whether existing transactions have the same type as their associated categories. Testing found no mismatches in the existing dataset. The endpoint reports inconsistencies but does not itself prevent invalid records from being inserted.

4. Reflection: Challenges and Lessons Learned

The development process highlighted the importance of consistency between frontend requests, backend responses, and database structures. Server-side pagination and interactive data retrieval

required careful handling of request parameters and response formats. Accurate summaries also depended on appropriate database queries and aggregation techniques. Testing individual API endpoints and dashboard features helped verify the flow of data through the application. Maintaining consistency between the local project and the GitHub repository was another important lesson, as clear source code and setup instructions improve reproducibility. The project strengthened practical understanding of full-stack development, relational database design, REST API integration, SQL calculations, asynchronous requests, and version control.

5. Future Enhancements and Conclusion

The core objectives from the conceptual design remained in place: two dashboard views, three financial visualizations, a relational database, a Flask API, and dynamic SQL-based summaries. Implementation refinements improved DataTables compatibility, transaction-count accuracy, and the ability to check transaction/category consistency. Future enhancements could include stronger validation when creating or modifying transactions, additional filtering options, data export, user authentication, and deployment to a hosted environment. Overall, the project demonstrates how frontend technologies, a Flask REST API, and MySQL can be integrated to retrieve, process, and present financial information. The finalization phase focuses on preparing the source code, setup instructions, and supporting documentation so the project can be reviewed and reproduced.